

AgroShield AI

Sistema Inteligente de Monitoramento Agrícola com Benchmark Computacional entre float64 e int8

Integrantes

- Gabriel Borba - RM553187
 - Gabriel Souza Fiore - RM553710
 - Gustavo Gouvêa Soares - RM553842
 - Pedro Henrique Mello Silva Alves - RM554223
 - Guilherme Santiago - RM552321
-

1. Introdução

A agricultura moderna enfrenta diversos desafios relacionados às mudanças climáticas, monitoramento do solo e dificuldade de acesso à tecnologia, principalmente entre pequenos e médios produtores rurais. Muitos produtores possuem dificuldades para interpretar dados técnicos e reagir rapidamente às alterações climáticas, o que pode gerar perdas financeiras, desperdício de água e redução da produtividade agrícola.

Diante desse cenário, o projeto AgroShield AI propõe o desenvolvimento de um sistema inteligente de monitoramento agrícola baseado em Inteligência Artificial e processamento embarcado. A solução utiliza uma rede neural MLP simples para analisar dados agrícolas e gerar inferências relacionadas ao risco agrícola.

Além disso, o projeto realiza um benchmark computacional entre os padrões numéricos float64 e int8, avaliando desempenho, precisão e tempo de processamento em sistemas operacionais preemptivos com Kernel 64 bits.

2. Questão de Pesquisa

Como diferentes padrões numéricos (float64 e int8) influenciam o desempenho computacional e a precisão de redes neurais embarcadas aplicadas ao monitoramento agrícola?

3. Objetivo Geral

Avaliar o impacto dos padrões numéricos float64 e int8 no treinamento de redes neurais MLP aplicadas ao monitoramento agrícola inteligente.

4. Objetivos Específicos

- Criar um dataset agrícola sintético;
 - Desenvolver uma rede neural MLP simples;
 - Realizar inferências agrícolas utilizando IA;
 - Comparar os padrões numéricos float64 e int8;
 - Medir tempo de processamento e precisão;
 - Analisar a influência dos sistemas operacionais preemptivos.
-

5. Dataset Utilizado

O dataset utilizado no projeto foi desenvolvido de forma sintética para simular cenários agrícolas reais. Os atributos utilizados foram:

- Umidade do solo;
 - Temperatura;
 - Índice de chuva;
 - Fertilidade do solo;
 - Radiação solar;
 - Risco agrícola.
-

6. Desenvolvimento da Rede Neural

A aplicação foi desenvolvida utilizando a linguagem Python juntamente com as bibliotecas TensorFlow, Pandas, NumPy, Matplotlib e Scikit-learn. A rede neural

implementada foi uma MLP simples contendo uma camada oculta e uma camada de saída com função de ativação sigmoideal.

O treinamento foi realizado utilizando epochs, accuracy e loss como métricas principais para avaliação do desempenho da rede neural.

```
> main.py > ...  
  
modelo_int8 = Sequential()  
  
modelo_int8.add(Dense(8, input_dim=5, activation='sigmoid'))  
  
modelo_int8.add(Dense(1, activation='sigmoid'))  
  
modelo_int8.compile(  
    loss='binary_crossentropy',  
    optimizer='adam',  
    metrics=['accuracy']  
)  
  
inicio_int8 = time.time()  
  
historico_int8 = modelo_int8.fit(  
    X_treino,  
    y_treino,  
    epochs=50,  
    verbose=0  
)  
  
fim_int8 = time.time()  
  
tempo_int8 = fim_int8 - inicio_int8  
  
loss_int8, acc_int8 = modelo_int8.evaluate(  
    X_teste,  
    y_teste,  
    verbose=0  
)  
  
# RESULTADOS  
  
print("\n===== RESULTADOS =====")  
  
print(f"\nFLOAT64")  
print(f"Accuracy: {acc_float64:.2f}")  
print(f"Loss: {loss_float64:.2f}")  
print(f"Tempo: {tempo_float64:.2f} segundos")  
  
print(f"\nINT8")  
print(f"Accuracy: {acc_int8:.2f}")  
print(f"Loss: {loss_int8:.2f}")  
print(f"Tempo: {tempo_int8:.2f} segundos")  
  
# GRÁFICO COMPARATIVO
```

7. Estratégia de Testes

Foram realizados testes relacionados ao desempenho computacional, precisão das inferências e tempo de processamento utilizando os padrões numéricos float64 e int8.

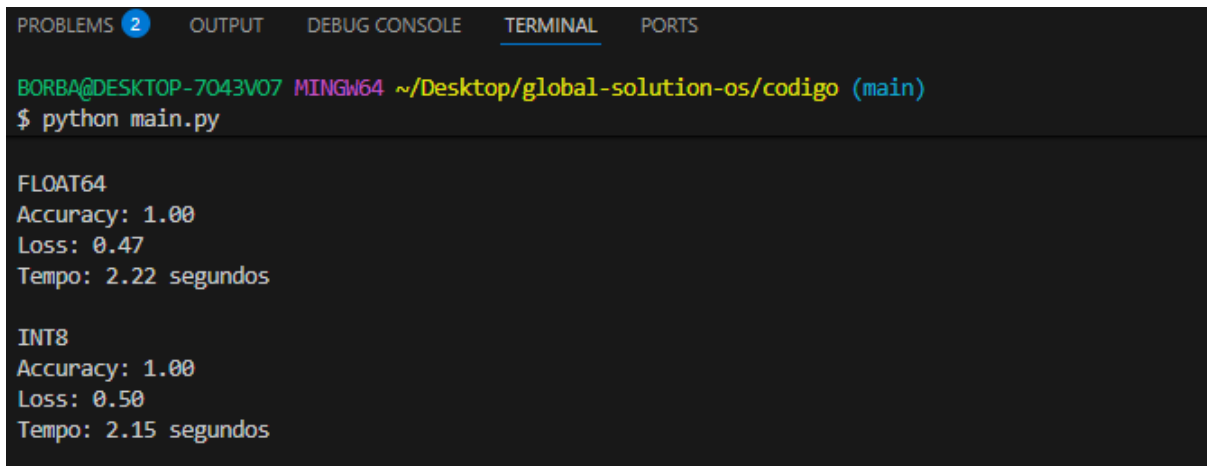
- Treinamento da rede neural;
- Avaliação de accuracy e loss;
- Benchmark computacional;

- Comparação entre float64 e int8;
 - Análise de tempo de processamento.
-

8. Resultados Obtidos

Os resultados demonstraram diferenças importantes entre os padrões numéricos utilizados. O padrão float64 apresentou maior precisão, porém maior custo computacional. Já o padrão int8 apresentou menor tempo de processamento e menor consumo computacional, sendo mais adequado para sistemas embarcados e aplicações em tempo real.

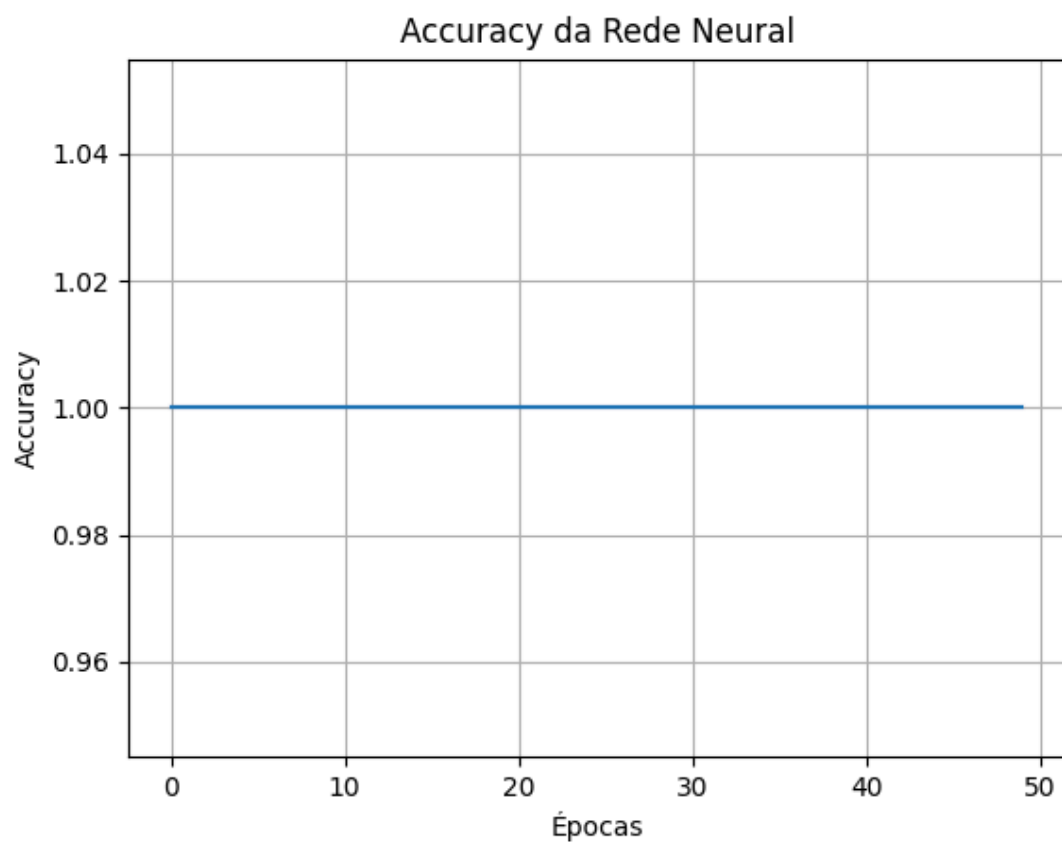
Os gráficos gerados durante os testes demonstraram a evolução do accuracy, loss e o benchmark de tempo de processamento entre os dois padrões numéricos.

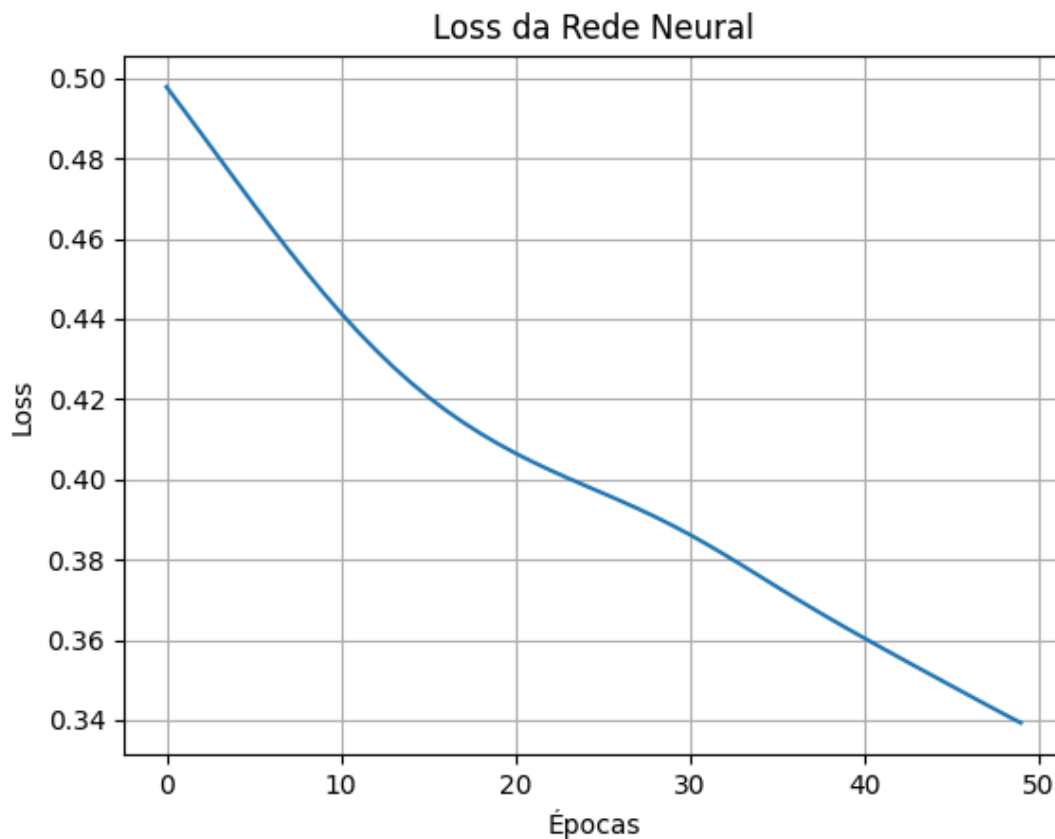


```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
BORBA@DESKTOP-7043V07 MINGW64 ~/Desktop/global-solution-os/codigo (main)
$ python main.py

FLOAT64
Accuracy: 1.00
Loss: 0.47
Tempo: 2.22 segundos

INT8
Accuracy: 1.00
Loss: 0.50
Tempo: 2.15 segundos
```





```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
BORBA@DESKTOP-7043V07 MINGW64 ~/Desktop/global-solution-os/codigo (main)
$ python main.py

FLOAT64
Accuracy: 1.00
Loss: 0.47
Tempo: 2.22 segundos

INT8
Accuracy: 1.00
Loss: 0.50
Tempo: 2.15 segundos
```

9. Análise de Sistemas Operacionais

Em sistemas operacionais preemptivos, múltiplos processos disputam recursos computacionais simultaneamente. Dessa forma, aplicações que utilizam tipos numéricos mais pesados, como float64, podem apresentar maior consumo de memória RAM e maior tempo de resposta.

Por outro lado, o padrão int8 reduz significativamente o custo computacional, sendo uma alternativa mais eficiente para aplicações embarcadas, Edge AI e sistemas críticos que necessitam de respostas rápidas e previsíveis.

Conforme observado nos testes realizados, o padrão int8 apresentou menor tempo de processamento em comparação ao float64, demonstrando maior eficiência computacional para aplicações embarcadas e sistemas operacionais preemptivos.

10. Conclusão

O projeto AgroShield AI demonstrou como Inteligência Artificial e sistemas embarcados podem ser aplicados no monitoramento agrícola inteligente. A comparação entre float64 e int8 evidenciou diferenças relevantes de desempenho e precisão computacional.

Além de atender aos objetivos propostos pela disciplina, o projeto também demonstrou como soluções de IA podem contribuir para agricultura de precisão, otimização de recursos e tomada de decisão em ambientes críticos.

11. Tecnologias Utilizadas

- Python
- TensorFlow
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- GitHub